

Chair of Computer Architecture (M. Sc.)

Model Enumeration using IPASIR-UP

Master's Thesis

First Examiner

Prof. Dr. Armin Biere

Second Examiner

Prof. Dr. xyz

Author

Timpe Horig

Matrikelnummer: 5108196

Geburtsort: Neustadt, Deutschland

Working Period

start date - end date

Abstract

Contents

1	Introduction	1
2	IPASIR	2
3	IPASIR-UP	3
4	CaDiCall	5
4.1	Conflict Driven Clause Learning	5
4.2	Chronological Backtracking	5
4.2.1	Backtracking after a model is found	6
4.2.2	Conflict induced backtracking	9
4.3	Deciding	9
4.4	Implicant Shrinking	10
5	Extending IPASIR-UP	14
5.1	Making IPASIR-UP the only interface needed	14
5.1.1	Backtracking after a model was found	14
5.1.2	Maintaining chronological backtracking	15
5.1.3	Out of order propagations	16
5.2	Implicant Shrinking	16
5.3	Optimization	18
5.3.1	Interacting with the solver	18
5.3.2	Duplicating Datastructures	18
5.3.3	Storing Models	18
6	Benchmarks	19
6.1	Configuration	19
6.2	Baseline	20
6.3	Benchmark Sets	20
6.4	Evaluation Metrics	20
6.5	Results	21
6.6	Benchmark Conclusion	26
7	Conclusion	27

1 Introduction

The Boolean satisfiability problem (SAT) is defined as follows:

Given a propositional formula F over variables $V = \{v_1, \dots, v_n\}$, decide whether there exists an assignment $\alpha : V \rightarrow \{0, 1\}$ such that $\alpha \models F$.

This problem was the first to be proven NP-complete by Cook [1] in 1971 and independently by Levin [2] in 1973. As a result, finding efficient algorithms for the SAT problem quickly became one of the most important and influential challenges in computer science. Modern Conflict-Driven Clause Learning (CDCL) solvers are capable of handling formulas with millions of variables and clauses. As a result, SAT solving plays a central role in hardware and software verification, automated reasoning, planning, scheduling, model checking, and cryptanalysis.

State-of-the-art solvers such as CaDiCal, Glucose, and Kissat demonstrate how theoretical advances combined with low level optimizations can transform an intractable worst-case problem into a powerful practical tool. These solvers typically focus on the decision problem: determining whether at least one satisfying assignment exists. If the formula is satisfiable, the solver returns a single model.

However, many applications require more than just one solution. In areas such as configuration, diagnosis, quantitative reasoning, explanation generation, and formal verification, it is often necessary to enumerate all satisfying assignments or a specific subset of them. This task, known as model enumeration (or All-SAT), generalizes the classical SAT problem from computing a single satisfying assignment to computing the whole set $\{\alpha : V \rightarrow \{0, 1\} \mid \alpha \models F\}$. While conceptually straightforward, model enumeration introduces significant algorithmic challenges. The number of satisfying assignments may be exponential in the number of variables, and naive approaches can quickly become infeasible.

A common technique for model enumeration is the iterative use of a SAT solver combined with blocking clauses. After each model is found, a clause composed of the negation of every literal in that model is added to the CNF formula, ensuring that the solver does not rediscover the same assignment. With the introduction of standardized interfaces such as IPASIR, SAT solvers can be seamlessly integrated into larger systems without the need to maintain the solver itself. More recently, IPASIR-UP was introduced, expanding IPASIR with User Propagators. This allows adding clauses without restarting the search. Using IPASIR-UP with Blocking Clauses yields a very clean model enumerator, that benefits from the underlying solver CaDiCal while CaDiCal itself remains unchanged. Although effective, this approach has several drawbacks. The repeated addition of blocking clauses increases the size of the formula, thereby degrading solver performance by affecting propagation efficiency and memory consumption. As the number of models grows, these effects scale up, eventually making blocking clauses infeasible.

In 2024 [3] presented TabularAllSat: A Sat solver that is especially designed for

disjoint projected enumeration for SAT and SMT without blocking clauses. TabularAllSat outperforms all other state-of-the-art AllSAT solvers. This was achieved by "combining the best of current All- SAT state-of-the-art literature": (i) Conflict Driven Clause Learning short CDCL, (ii) Chronological backtracking to avoid blocking clauses and (iii) Implicant shrinking to minimize models.

Besides adding clauses iteratively, IPASIR-UP also allows to interfere with decision making, backtracking and found models. This raises the question whether it is possible to create a model enumerator through IPASIR-UP without blocking clauses by using all state-of-the-art techniques from TabularAllSat.

This thesis introduces CaDiCaL, a model enumerator which is almost entirely implemented through the IPASIR-UP interface and does not need any blocking clauses. "Almost" because there are two minor additions that need to be added to IPASIR-UP: One is already added to the newest version of CaDiCaL's IPASIR-UP interface and the other is fixing a bug introduced by the former. As IPASIR-UP is an interface for CaDiCaL and CaDiCaL being a CDCL-based solver, technique (i) from TabularAllSat is naturally covered. Also technique (ii) chronological backtracking is fully possible through the IPASIR-UP interface and thus also used in CaDiCaL. Using (iii) the implicant shrinking algorithm proposed by TabularAllSat requires access to the watch-lists of literals. This is not supported by the IPASIR-UP interface. However, it is still implemented in CaDiCaL and Section 5 discusses which additional functionality is needed and how it could be added to the IPASIR-UP interface.

CaDiCaL supports a fixed variable order. However, per default CaDiCaL decides. It also can be used as a simple model counter, which makes it possible to count models for formulas with exponential many models while keeping memory consumption very little. Furthermore, as IPASIR-UP also introduces the possibility to propagate literals. It is also possible to backtrack through adding reason clauses if necessary.

2 IPASIR

The IPASIR interface standardizes incremental interaction with CDCL-based SAT solvers by defining a minimal set of functions through which the user can call the solver in bigger contexts while keeping the program Independent of the solver. A solver instance is created and subsequently manipulated only via functions such as `ipasir_add` to add literals to the current clause, terminated by 0. `ipasir_assume` to add temporary assumption literals for the next call to `ipasir_solve`, and `ipasir_solve` to start or resume the search. After `ipasir_solve` returns SAT or UNSAT, the user may query models with `ipasir_val` or inspect a failed core via `ipasir_failed`. The interface deliberately abstracts from internal solver mechanisms: There is no access

to decision levels, propagation events, learned clauses, or backtracking. Although learned clauses are preserved across calls in incremental mode, the user cannot intervene during the search itself. Consequently, model enumeration requires blocking clauses: After finding a model, add its negation as a new clause to the cnf (via `ipasir_add`) and restart the solver. If the solver returns UNSAT, all models are found. However, this approach is sub-optimal as each new blocking clause increases the cnf. This makes it infeasible for big CNFs with many models. Model enumeration without blocking clauses through IPASIR is impossible since the solver terminates immediately upon finding a satisfying assignment.

3 IPASIR-UP

To overcome the limitations from IPASIR, Katalin Fazekas et al. introduced IPASIR-UP in 2023 as an extension of IPASIR that introduces **user propagators** (UP). IPASIR-UP augments the classical interface with a bidirectional communication layer between the solver’s CDCL loop and user-defined callbacks. Instead of restricting interaction to clause addition and top-level solve calls, IPASIR-UP enables controlled intervention during propagation, decision making, backtracking, and model detection.

- Model validation

```
bool cb_check_found_model(const tmodel &model) override {
    return true;
}
```

This function is called whenever the solver derives a total satisfying assignment. If the user propagator returns `true`, the solver reports SAT as usual. If it returns `false`, the model is rejected and the solver continues the search without restarting. Crucially, the internal trail, learned clauses, and heuristic state remain intact.

- External clause injection

```
bool cb_has_external_clause(bool &is_forgettable) override {
    return false;
}

int cb_add_external_clause_lit() override { return 0; }
```

These functions allow the user propagator to provide clauses to the solver right after `cb_check_found_model` was called. When `cb_has_external_clause` returns `true`, the solver repeatedly calls `cb_add_external_clause_lit` to obtain the literals of the clause (terminated by 0).

- Notifications

```
void notify_assignment(const ivec &list) override {}
void notify_new_decision_level() override {}
void notify_backtrack(size_t new_level) override {}
```

These notification callbacks inform the user propagator about changes to the assignment trail. `notify_assignment` communicates newly assigned literals, `notify_new_decision_level` signals that a decision was made and thus the decision level increased, and `notify_backtrack` reports to which level the solver backtracked.

- Decision making

```
int cb_decide() override { return 0; }
```

This callback is invoked before the solver makes the next decision. If it returns a non-zero literal, the solver uses it as the next decision. If 0 is returned, the solver makes its own decision, based on its internal decision heuristic.

- User propagation

```
int cb_propagate() override { return 0; }
int cb_add_reason_clause_lit(int propagated_lit) override {
    return 0;
}
```

`cb_propagate` enables the user propagator to return implied literals under the current partial assignment. If a conflict is detected in which the propagated literal was involved, the solver asks for a reason clause through repeatedly calling `cb_add_reason_clause_lit` (terminated by 0). This enables steering through the decision space while adding the reason clauses only if really necessary.

A user must define a User Propagator defining all the functions above.

```
void connect_external_propagator (ExternalPropagator* propagator);
void disconnect_external_propagator ();
void add_observed_var (int var);
void remove_observed_var (int var);
```

Then User Propagator can simply be connected and disconnected to the solver by calling `connect_external_propagator` and `disconnect_external_propagator`.

The notification callback functions are only called for variables marked as `observed`. The user can mark and unmark variables through the functions `add_observed_var` and `remove_observed_var`.

4 CaDiCAll

CaDiCAll is a fully functional model enumerator using CaDiCaL almost exclusively through the IPASIR-UP interface. It is implemented by using all three state-of-the-art mechanisms defined by Spallitta et al[3]: Conflict Driven Clause Learning (Section 4.1), Chronological Backtracking (Section 4.2) and Implicant Shrinking (Section 4.4).

4.1 Conflict Driven Clause Learning

Conflict-Driven Clause Learning (CDCL) improves the classical DPLL algorithm by learning from conflicts. When the solver runs into a conflict, it does not simply flip the last decision but analyses the conflict to determine the assignments that caused the contradiction.

The analysis starts with the conflicting clause (the clause that is falsified by the current assignment). The solver inspects which literals in that conflict clause were forced by another clause and resolves them with each other. Let the conflict clause be $(A \vee \neg x_1)$ and x_1 be forced by $(B \vee x_1)$. Resolving these two clauses on x_1 yields the reason clause $(A \vee B)$. This process is repeated until a learned clause of the form $(A \vee B \vee \dots)$ is obtained that contains exactly one literal from the current decision level, which is then added to the CNF. Such a clause enables non-chronological backtracking: the solver backtracks to the second highest decision level occurring in the clause. At that point, all other literals in the clause remain falsified while the literal from the previous decision level becomes the only unassigned one, making the clause unit and forcing a new assignment through unit propagation. This prevents the solver from encountering the same conflict again.

By learning new clauses and pruning conflicting regions of the search space, CDCL significantly improves the efficiency of SAT solving. For this reason, it forms the basis of most modern SAT solvers.

As CaDiCaL is a CDCL-based solver, CDCL is implicitly used by CaDiCAll as it uses CaDiCaL through the IPASIR-UP interface.

4.2 Chronological Backtracking

Model enumeration counts every model exactly once. To do so, we must ensure, that paths which lead to models are excluded from future exploration. In common model enumerators this is done by blocking clauses, which permit the solver to run

into the same model again. Without blocking clauses we must ensure manually that every path is explored exactly once.

Modern SAT solvers based on Conflict-Driven Clause Learning (CDCL) employ non-chronological backtracking (NCB). During conflict analysis, the solver may determine that some earlier decisions did not contribute to the conflict and can therefore backtrack across multiple decision levels at once. By skipping parts of the search tree that cannot lead to a solution, this approach significantly improves efficiency, but it also changes the order in which decisions are revisited and effectively reshapes the search tree during the search. NCB can lead to redundant work, which is not a problem in normal Sat, as the solver stops after finding one model and thus rediscovering the same model is not a problem. However, when enumerating all models, the solver does not terminate after finding the first model. Instead, it must check the remaining search space for others. As NCB can alter the search space and does not uncover the search space systematically, the only possibility to prevent the solver from rediscovering the same model again is to exclude the exact combination of literals in this model. Implying blocking clauses.

Exploring the search space systematically makes blocking clauses redundant. The idea is simple: When backtracking, always only backtrack to the last decision and flip it. If there is no more decision to flip, we can terminate as the whole search space is uncovered. This is known as Chronological Backtracking (CB). With CB all currently made decisions split the search space into two parts: The part that already has been uncovered and the part that is left to explore. With this approach it suffices to keep track of the current decisions instead of all models that have already been explored.

Transferring this idea to CaDiCaL imposes different challenges: First, backtracking after a model is found was not supported by the IPASIR-UP interface. Second, even though chronological backtracking can be set as an option in CaDiCaL. CaDiCaL does not guarantee to assign the negation of the old decision. Thus, to keep track of the current decisions we store them. As discussed in Section 4.3 the decision order as well as the phase of decisions can be chosen freely and is left for CaDiCaL to decide. Thus, in order to keep track of the current decisions, for each decision level we store a counter (decision counter) and the decided variable. The decision counter indicates how often this variable was assigned. A count of one stands for a new decision while a count of two indicates that this decision was already flipped. When CaDiCaL determines the highest decision level with a decision that can be flipped, it simply looks for the highest decision level with a decision count of one.

4.2.1 Backtracking after a model is found

Backtracking after a model is found was not supported by the IPASIR-UP interface. However, as discussed in Section 5 it is fairly easy to add and does not introduce

any bad behavior. Thus, allowing to force CaDiCaL to backtrack when a model is found was added to the newest version of CaDiCaL’s IPASIR-UP interface by Katalin Fazekas.

When a model is found, CaDiCaL determines the highest decision level `d1` with a decision count of one and then forces CaDiCaL to backtrack to the decision level `d1 - 1`. If this level is negative there exists no decision that has not been already flipped and the solver can terminate as there is no path left to explore. Otherwise, the decision on level `d1` must be flipped. This is not done automatically. CaDiCaL has no reason to flip it as the backtrack did not occur from a conflict but artificially from the user propagator.

There are two different options to continue the search from here: Either propagate the negation of the decision at level `d1 - 1` via `cb_propagate` or decide the negation of the decision at level `d1` via `cb_decide`. Both versions are implemented in CaDiCaL.

Continue the search with propagation Continuing the search through propagating the negation of the decision via `cb_propagate` seems elegant. When the solver backtracks later again, it does not need to consider flipped decisions as they were propagated and thereby are no decisions anymore. However, if the propagated literal is involved in a conflict and inspected in a conflict analysis, the solver asks for a reason clause via `cb_add_reason_clause_lit` for the propagated literal. The reason clause for this literal is just the negation of all decisions made on a lower level. Thus, reason clauses must not be stored but can be computed on the fly only if necessary. This bears two advantages to blocking clauses: The size of the reason clause is smaller than the size of blocking clauses as it only contains decisions on a lower level and the reason clauses are only added if necessary.

However, after forcing CaDiCaL to backtrack when a model was found, CaDiCaL may have learned that it can keep certain literals from higher levels. This may trigger propagation, which can lead to different behavior. Let the old decision be `d`. There are four cases to consider:

1. The solver propagates the old decision `d` again. In this case the solver starts to explore the same path again. This is detected directly in `notify_assignment`. However, the first it can be handled are either by `cb_decide` or `cb_check_found_model` as these are the only two functions that can force the solver to backtrack. The function `notify_assignment` marks the current path as a duplicate and as soon as either `cb_decide` or `cb_check_found_model` is called, it forces the solver to backtrack further. This is sound as everything that led to the propagation of `d` was propagated itself. Thus, the propagation of `d` is a direct consequence of the decision on the current level.
2. The solver runs into a conflict while propagating. In this case, a conflict

induced backtracking occurs. This is described in the Section 4.2.2.

3. The solver only propagates literals which are different from d or non at all. In that case, when `cb_propagate` is called, $\neg d$ will be propagated.
4. The solver propagates $\neg d$. CaDiCaL will still try to propagate $\neg d$ by returning it in `cb_decide`. However, CaDiCaL will ignore this as it is already assigned and continue.

While this approach does optimize blocking clauses, it still uses them. Thus, CaDiCaL focuses on the second approach, which does not require adding any clauses at all. Running CaDiCaL with the first approach is still possible by running it with the option `--reason (-r)` and is also included in the benchmarks in Section 6.

Continue the search with decisions Continuing the search through deciding the negation of the decision via `cb_decide` does not need any clause addition. After determining the highest decision level `d1` with a count of one. CaDiCaL backtracks to the decision level `d1 - 1`, then it decides the negation of the old decision and continues from there. However, before `cb_decide` is called and CaDiCaL can decide the negation of the old decision, CaDiCaL propagates which again can lead to different behavior. Let d be the old decision. The following four cases must be considered:

1. The solver propagates the old decision d . Thus, the solver starts to rediscover the old path. This is again detected directly in `notify_assignment` but can be handled the earliest when either `cb_decide` or `cb_check_found_model` is called. There, the solver is forced to backtrack further.
2. The solver runs into a conflict while propagating. In this case, a conflict induced backtracking occurs. This is described in Section 4.2.2.
3. The solver only propagates variables that are different to decision d or none at all. In that case the next time `cb_decide` is called, CaDiCaL detects that on the current level there already has been a decision earlier and decides its negation by return it.
4. The solver propagates $\neg d$ itself. Then the user propagator does not need to decide it manually and can let the solver chose the next decision itself by returning 0 in `cb_decide`.

Here it is important to note that now decision can have a decision counter of two. This is not a problem when a model is found and the user propagator forces the solver to backtrack, as the decision level to backtrack to is determined by the user propagator as well. However, when a conflict occurs and CaDiCaL backtracks by itself, this must be considered.

4.2.2 Conflict induced backtracking

When CaDiCaL runs into a conflict and backtracks, it is handled completely by CaDiCaL internally. It informs the user propagator via `notify_backtrack` but does not allow any interference. With chronological backtracking activated, we know that CaDiCaL backtracks exactly one level as it always backtracks to the last decision. However, when the search was continued with decision and not through propagating the negation of the old decision. The last decision might be one with a decision count of two. Here, CaDiCaL will still backtrack to this decision, and continue the search from there. In that case, the solver rediscovers the old search space and everything the solver does from this moment on must be undone again. This case is detected by CaDiCaL as it notices that the decision count on this level is already two and forces CaDiCaL to backtrack further. Unfortunately, this can lead to an overhead as the solver may spend some time propagating before the user propagator can interfere.

Otherwise, we know that the decision count of the level we backtracked to is one and the search can continue from here. Even though CaDiCaL derived a learned clause from the conflict, it does not necessarily learn the negation of the old decision and thus, the same four cases as when backtracking after a model was found can occur:

1. The solver propagates the old decision `d`. Thus, the solver starts to rediscover the old path. This is again detected directly in `notify_assignment` but can be handled the earliest when either `cb_decide` or `cb_check_found_model` is called. There, the solver is forced to backtrack further.
2. The solver runs into a conflict while propagating. In this case, a conflict induced backtracking occurs again.
3. The solver only propagates variables that are different to decision `d` or none at all. In that case the user propagator can continue normally and return the flipped decision in `cd_decide`.
4. The solver propagates $\neg d$ itself. Then the user propagator does not need to decide it manually and can let the solver chose the next decision itself by returning 0 in `cb_decide`.

4.3 Deciding

The order in which decisions are made can heavily impact the performance of sat solvers. The right decision can lead the solver into early conflicts, pruning big parts of the search space completely or skipping long nested parts. While in model enumeration the later benefit is lost as all models must be found, it still profits from early conflicts.

In order to search systematically through the whole search tree, a fixed variable order is the easiest. On each `cb_decide` call, decide the next undecided variable positively. If the solver needs to backtrack, simply find the highest decision level with a positive decision. However, this constraint can be weakened while still preserving a systematically order.

First, the phase of a decided variable can be chosen freely, while keeping backtracking still simple enough. For each decision level a decision count is saved. Instead of backtracking to the highest decision level with a positive decision, simply backtrack to the highest decision level with a decision count of 1. Meaning it has been decided exactly once.

Second, the order in which variables are decided can also be chosen complete free, as long as after a backtracking the decision is just flipped. This enables the solver to profit from dynamic decision heuristics such as Variable State Independent Decaying Sum (VISIDS) which is known to improve solver performance significantly and thus is a commonly used decision heuristic in modern sat solvers. The performance difference between fixed and dynamic decision orders can be seen in Section 6.

The decision count is updated if a decision is made or flipped and reset if the decision is propagated or backtracked over. If the solver asks for a decision and the decision count on the current level is one. The decision must be flipped and thus `cb_decide` returns the negation of the old decision. Otherwise, `cb_decide` returns 0 and thus, lets the solver decide itself.

4.4 Implicant Shrinking

Models (also called implicants) found by the solver are total. Models are total if they contain every variable. For a CNF $(x_1 \vee x_2) \wedge (x_1 \vee \neg x_2)$ a model enumerator returns two total models: $x_1 \wedge x_2$ and $x_1 \wedge \neg x_2$. As x_1 occurs in both models positively, the only difference lays in x_2 which occurs once positively and once negatively. In other words, if x_1 is positiv, it does not matter what the value of x_2 is, it is always a model for the CNF. Thus, these models can be merged into one model by dropping x_2 , resulting in one partial model x_1 . This reduces both, the amount of models in total and the size of models. It also may fasten the solving itself. Normally, after finding the first model $x_1 \wedge x_2$ a model enumerator backtracks to the last decision that can be flipped. Assuming the decisions where x_1 at decision level 1 and x_2 at decision level 2, a model enumerator would backtrack to decision level one and then continue with $\neg x_2$. However, if the model enumerator shrinks the model $x_1 \wedge x_2$ to x_1 the last decision that can be flipped is x_1 . Thus, the model enumerator backtracks to decision level 0 and can continue with $\neg x_1$, skipping the branch with $\neg x_2$ completely. As the branch containing x_2 could be arbitrary large, this may reduce the search space significantly.

Finding out which literals can be dropped is non trivial. Implicant shrinking

algorithms from non chronological backtracking based model enumerators do not guarantee mutual exclusiveness between different assignments[3]. Spallitta et al. [3] introduces their own shrinking algorithm that is both: Compatible with chronological backtracking and that tries to cut the highest amount of literals while still ensuring mutual exclusivity. This algorithm starts with the last literal on the Trail and tries to drop as many literals as possible. A literal ℓ can be dropped if ℓ is a decision and all clauses in which ℓ occurs contain another literal ℓ' that satisfies the clause.

```
int implicant_shrinking(vector<int> stack) {
    int b = 0;
    int index = stack.size() - 1;
    while (index >= 0) {
        int v = stack[index];
        if (!decision(v)) {
            b = std::max(b, decision_level(v));
        } else if (decision_level(v) > b) {
            b = check_literal(v, b, stack, index);
        } else if (decision_level(v) == 0 || decision_level(v) == b) {
            break;
        }
        index--;
    }
    return b;
}
```

In CaDiCaL the 2-watched-literal scheme is used. This allows the solver not to constantly check all clauses if the assignment of a literal changes. Instead every clause of the CNF is only watched by two literals, let them be ℓ_1 and ℓ_2 . This is enough to track the status of the clause if the following invariant holds: At least one literal is not falsified. In order to keep this invariant alive, if one watched literal ℓ_1 becomes falsified the solver tries to find a replacement. If there is another literal ℓ' in that clause that is neither false nor ℓ_2 , it replaces ℓ_1 as the watching literal. If there exists no such literal ℓ' the solver knows that ℓ_2 is the only literal left that could satisfy the clause and forces ℓ_2 . If both literals ℓ_1 and ℓ_2 are falsified, the clause is falsified, yielding UNSAT.

If we want to drop a literal ℓ , it suffices to check all clauses that are watched by ℓ instead of all clauses in which ℓ occurs. If a model is found, we now at least one of the watching literals for every clauses is satisfied. Thus, dropping a literal that is not watched will never falsify a clause. However, if we drop a literal ℓ that is currently watching a clause c , we know there exists another literal $\ell' \in c$ that satisfies c . This literal ℓ' must now become the new watching literal for c . Otherwise, when trying

to drop ℓ' , c would not be checked.

Going through all literals in all clauses watched by ℓ may introduce a bottleneck. Thus Spallitta et al [3] introduced a faster but more conservative algorithm exploiting the 2-watched-literal scheme. Conservative meaning it may not drop all literals that could be dropped.

The 2-watched-literal scheme is used in the the faster approach proposed from Spallitta et al. [3] as shown in algorithm 1.

Algorithm 1 CHECK-LITERAL(ℓ, b, T') (faster but more conservative)

```

1: for all  $c \in \omega(\ell)$  do
2:    $\ell' \leftarrow$  the other watched literal in  $c$ 
3:   if  $\ell' \in T'$  then
4:     Continue
5:   else
6:      $b \leftarrow \max(b, \delta(\ell))$ 
7:   end if
8: end for
9: return  $b$ 

```

```

int check_literal(int v, int b, vector<int> stack, int index) {
    for (int lit : other_watching_literals(v)) {
        if (!satisfied_by(lit, get_n(stack, index))) {
            b = std::max(b, decision_level(v));
        }
    }
    return b;
}

```

Instead of checking all other literals in the watched clause, simply check the other literal ℓ' that watches that clause. If it satisfies the clause as well, the literal ℓ can be dropped. As ℓ' already watches the clause, nothing must be updated. This is more conservative as it may not drop literals that could be dropped. If ℓ' is falsified but another literal in that clause is satisfied, ℓ still could be dropped but is not. However, as this conservative approach does not need to traverse the clause and check each literal, it is significantly faster.

As IPASIR-UP does not allow any access to watches, this is implemented by using a backdoor to CaDiCaL directly. Section 5 discussed what is needed and how it could be added to the IPASIR-UP interface.

Adapting this implicant shrinking algorithm to CaDiCaL using IPASIR-UP may introduce a fault. As stated in Section 4 there are two different ways to flip a decision. If the decision is flipped by deciding its negation, ?? must be adapted.

Consider the CNF $(x_1 \vee x_2 \vee x_3)$. Let x_2 and x_3 be the watched literals at the start and the decision order be $x_1 \rightarrow x_2 \rightarrow x_3$. The first model to be found will be

$x_1 \wedge x_2 \wedge x_3$. Implicant shrinking will first remove x_3 as the other watched literal (x_2) still satisfies the clause. Then it will try to remove x_2 . However, as the other watched literal is x_3 which is no longer on the trail, it cannot continue. Thus, discovering the model $x_1 \wedge x_2$. Then the solver backtracks and flips x_2 by deciding its negation $\neg x_2$. Remind that $\neg x_2$ is still a decision. As x_2 is one of the watched literals but is now falsified, x_1 becomes the new watched literal, as it is the only literal in the clause that is neither false nor already watched. Then the solver decides x_3 resulting in the model $x_1 \wedge \neg x_2 \wedge x_3$. Now implicant shrinking starts again. First removing x_3 as the other watched literal (x_1) still satisfied the clause. Next it tries to remove $\neg x_2$ as it is the next highest decision. As $\neg x_2$ does not occur in any clauses, it cannot be a watched literal. Thus removing it does not falsify any clause. Then, it tries to remove x_1 . However, as the other watched literal is x_3 which was already removed, x_1 can not be removed. Hence, yielding the model x_1 .

The implicant shrinking algorithm yielded both models $x_1 \wedge x_2$ and x_1 . These models are not distinct, as setting x_2 in the later generates the former.

This can be fixed by allowing implicant shrinking only to remove literals with a decision count of 1. Consider a variable with decision count 2 to be checked by implicant shrinking. There are two possibilities: First, the variable could not be removed when it's decision count was 1. Removing it now could trigger exactly a case like the example above. Second, the variable was removed when it's decision count was 1. In that case the model enumerator would have backtracked over the variable and thus resetting the decision count, which makes this case impossible to reach. Resulting in the following fix for the implicant shrinking algorithm:

```
int implicant_shrinking(vector<int> stack) {

    int b = 0;
    int index = stack.size() - 1;
    while (index >= 0) {
        int v = stack[index];
        if (!decision(v) || (decision(v) && decision_count(v) == 2)) {
            b = std::max(b, decision_level(v));
        } else if (decision_level(v) > b) {
            b = check_literal(v, b, stack, index);
        } else if (decision_level(v) == 0 || decision_level(v) == b) {
            break;
        }
        index--;
    }

    return b;
}
```

Flipping decisions by propagation their negations (Section 4) is not affected by this fix, as the highest decision count is always one.

5 Extending IPASIR-UP

The core idea underlying CaDiCaL is to use IPASIR-UP as the sole interface to the underlying SAT solver. Consequently, all functionality required for model enumeration must be realized exclusively through this interface. This design choice offers a clear advantage: CaDiCaL can be readily adapted to any solver that supports IPASIR-UP, without requiring modifications to the solver itself. However, IPASIR-UP in its current form does not provide all the functionality necessary for efficient model enumeration. As a result, certain extensions to the interface are required to achieve competitive performance.

We propose three categories of extensions to IPASIR-UP. First, we introduce the features necessary to enable model enumeration through the interface. Second, additional features that allow implicant shrinking. Third, we point out where duplication due to IPASIR-UP occur and how to reduce it and thereby improving the overall runtime performance of CaDiCaL.

5.1 Making IPASIR-UP the only interface needed

Most changes only affect the user propagator and not the solver directly. However, there are some features of CaDiCaL that break the chronological order of variables and thus must be taken into account.

5.1.1 Backtracking after a model was found

In conventional SAT solving, backtracking only occurs when the solver encounters a conflict. To continue the search, the solver must undo at least one previous decision and explore a different branch of the search space.

However, a similar situation arises when enumerating all satisfying assignments (models). Once the solver finds a model, the current assignment satisfies all clauses and therefore no conflict is present to trigger backtracking. Nevertheless, the search must continue in order to determine whether additional models exist. Thus, the solver must be forced to leave the current branch of the search space and explore alternative assignments.

When a model is found, IPASIR-UP allows the user propagator to observe and reject the model by returning `false` in `cb_check_found_model`. It then calls `cb_has_external_clause`. If the user propagator returns `true`, the solver repeatedly calls `cb_add_external_clause_lit` allowing the user propagator to add the

negation of the model as a blocking clause. This forces the solver to backtrack and explore other branches of the search space. However, as we do not want to add blocking clauses, CaDiCAll always returns `false` in `cb_check_found_model`. Which means, the solver is not forced to backtrack and would report the same model directly again. Resulting in an infinite loop where the solver is stuck.

In order to allow the user propagator to maintain additional constraints, IPASIR-UP provides the function `force_backtrack`. This function is allowed to be called inside `cb_decide`. It enables the user propagator to force the solver to backtrack to a specific decision level before making a new decision. Forcing the solver to backtrack inside `cb_decide` does not introduce bad behavior as at this point, the trail is fully consistent: All assignments up to the current decision level do not falsify the CNF, they are not in conflict with any other propagations and no propagations are pending.

When calling `cb_check_found_model` the trail is in an equally consistent state as in `cb_decide`: The trail fully satisfies the CNF and as it is a total assignment, there are no pending propagations. Thus, allowing `force_backtrack` in `cb_check_found_model` would not introduce any bad behavior.

To allow `force_backtrack` to be called from within `cb_check_found_model` simply the flag `forced_backt_allowed` must be set to `true` in the solver before calling `cb_check_found_model`.

```
forced_backt_allowed = true;
bool is_consistent = external->propagator->cb_check_found_model (etrail);
forced_backt_allowed = false;
```

This feature was added to the newest version of CaDiCaL's IPASIR-UP interface by Katalin Fazekas. Thus, allowing CaDiCAll to backtrack to the last decision after a model is found and continuing the search from there without adding blocking clauses.

5.1.2 Maintaining chronological backtracking

When the solver is about to make the next decision, the solver internally calls `ask_decision`, which then calls `cb_decide`. The user propagator is allowed to call `force_backtrack` inside `cb_decide`. However, backtracking might make the decision returned by the user propagator invalid, as it might be already assigned due to propagations after backtracking. However, instead of asking the user propagator again for the next decision, the solver makes the decision on its own. This imposes a problem as CaDiCAll does not always allow decision to be made by the solver, some decisions may be already fixed in order to force the solver to explore the search space chronologically. This can be fixed by calling `ask_decision` recursively instead of letting the solver decide. This way, the user propagator can make a new decision

after backtracking and thus keep the exploration of the search space chronological.

An important note is that, if the user propagator backtracks below the pseudo decision level of the solver. The solver goes back to the CDCL loop instead of making a decision. In that case the solver ignores the decision made by the user propagator, continues with the CDCL loop and calls `ask_decision` again when it is about to make the next decision. However, the CDCL loop may propagate literals that lead to paths that have already been explored. Thus, the user propagator must check whether the work done by the CDCL loop has led to such a path and if so, backtrack further. If not, the user propagator can either return the original decision or make a new one if the CDCL loop already propagated it.

```
if ((size_t) level < assumptions.size ()
    ((size_t) level == assumptions.size () && constraint.size ())) {
    return 0;
} else {
    return ask_decision();
};
```

5.1.3 Out of order propagations

In order to keep blocking literals correct, CaDiCaL propagates units at the root level to derive all implied units. However, this allows the solver to backtrack multiple levels and thus, skipping branches that potentially lead to models.

```
bool Internal::propagate_out_of_order_units () {
    return true;
};
```

5.2 Implicant Shrinking

The implicant shrinking algorithm described in section 4.4 needs access to the watchlists of a given literal in order to determine whether the other watched literal of a clause satisfies it or not. However, this is not supported via the IPASIR-UP interface. The implicant shrinking algorithm uses them read only. Thus, exposing a read only reference to them is unproblematic and could be integrated into IPASIR-UP without any changes to CaDiCaL itself. Here it is important to note that literals can be named differently external and internal. However, there already exist functions to translate from internal to external and vice versa, and thus the internal representation can be completely hidden through the IPASIR-UP interface.

A function implementing this feature would look like the following. It takes an external variable as an input and returns all other literals that are currently watching

the same clauses.

```
std::vector<int> other_watching_literals(int external_variable) {
    int internal_variable = internal->external->e2i[external_variable];
    int internal_literal = internal->external->vals[internal_variable] ?
        ↪ internal_variable : -internal_variable;

    std::vector<int> other_watching_literals;
    for (CaDiCaL::Watch watch : internal->watches(internal_literal)) {
        int i_wl1 = watch.clause->literals[0];
        int i_wl2 = watch.clause->literals[1];
        int i_other_lit = internal_literal == i_wl1 ? i_wl2 : i_wl1;
        int e_other_lit = internal->externalize(i_other_lit);
        other_watching_literals.push_back(e_other_lit);
    }

    return other_watching_literals;
}
```

The user propagator can then check whether all other watching literals are satisfied by the remaining trail. If so, the literal can be dropped without falsifying any clause. If not, the literal cannot be dropped as it is still needed to satisfy at least one clause.

If the sole purpose of this function is to be used by the implicant shrinking algorithm implemented from CaDiCaL, it could be optimized by checking if the other watching literal satisfies the clause instead of appending it to the list of other watching literals. This way, the function can return a boolean value indicating whether the literal can be dropped or not, without the need to return all other watching literals. This also allows early breaks, as the function can return false as soon as it finds a clause that is not satisfied by the other watching literal. However, the function would also need either access to the internal stack and position of literals, as the user propagator needs to check if the other watching literal is satisfied by the remaining trail, or it would need to be passed the current trail as an argument.

With this minor addition to the IPASIR-UP interface, the implicant shrinking algorithm can be implemented entirely through IPASIR-UP.

Currently, CaDiCaL implements this functionality by using this backdoor directly in the source code of CaDiCaL as it is not added to the IPASIR-UP interface yet.

5.3 Optimization

The primary goal of this thesis was to investigate whether and how a model enumerator can be implemented using IPASIR-UP. Optimizing CaDiCall and the IPASIR-UP interface to minimize runtime was not the main focus of this work. Nevertheless, while implementing CaDiCall, we identified some bottlenecks that could be optimized by adapting the IPASIR-UP interface.

5.3.1 Interacting with the solver

When the solver calls the functions `notify_assignments` and `cb_check_found_models` it creates a vector containing assigned literals. When we timed CaDiCall, in CNFs with many models, CaDiCall spend around 40 percent of its total time with vector operations. We found that when the solver is passing these vectors to the user propagator, the solver allocates new memory for each vectors and frees it afterwards again. By making these vectors persistent, the total running time of CaDiCall on CNFs with many models was cut in half.

5.3.2 Duplicating Datastructures

The most obvious performance bottleneck is the duplication of data structures.

In order to check where to backtrack to, if the solver rediscovers paths that have already been explored, or to check which literals can be dropped during implicant shrinking, CaDiCall needs to maintain the values of variables, their decision levels and whether they are decisions or propagations. IPASIR-UP provides enough information to observe all these information through the notification callback functions `notify_assignment`, `notify_new_decision_level` and `notify_backtrack`. However, this means that every time a variable is assigned, it is not only stored internally by CaDiCaL but also externally by CaDiCall. This does not only double the expenses but introduces an even bigger overhead as the information must also be passed from the solver to the user propagator through the callback functions. Thus, allowing read only access to CaDiCaLs variable assignment and stack would make most external representation redundant. As this would be a bigger change to the IPASIR-UP interface and is not necessary to implement a model enumerator, it is not implemented in the current version of CaDiCall. However, it could be added in the future to allow for more efficient model enumeration and other applications of IPASIR-UP as well.

5.3.3 Storing Models

Enumerating all models does not only require exponential time in the worst case but can also require exponential space as CNFs can have an exponential number of models. Thus, storing all models in memory may not be feasible for large CNFs.

Thus, CaDiCAll provides the option to only count models without saving them. This can be enabled by the command line option `--count` or `-c`.

6 Benchmarks

The goal of the experimental evaluation is to assess the performance of CaDiCAll and determine whether or not implementing a model enumerator using IPASIR-UP is feasible. In particular, we aim to evaluate:

- The impact of implicant shrinking
- The effectiveness of reason-based propagation
- The influence of a fixed variable order
- The overall competitiveness compared to TabularAllSat

6.1 Configuration

CaDiCAll supports multiple configurations, controlled via the command-line options:

- Shrinking (`--shrink` or `-s`)

Enables implicant shrinking using the algorithm described in the Section 4.4. This reduces models to smaller implicants, potentially finding models without actually computing them.

- Reason-based propagation (`--reason` or `-r`)

Instead of deciding the negation of decision literals, this mode propagates them using reason clauses. This can lead to faster backtracking as the solver can learn from the reason clauses and thus, may skip more of the search space. However, it also introduces an overhead as the solver needs to propagate the reason clauses and thus, may spend more time on propagations than on finding models.

- Fixed variable order (`--fixed` or `-f`)

Enabling this option forces the solver to explore the search space in a fixed order. This is achieved by always deciding the next undecided variable in a fixed order. This allows CaDiCAll to systematically explore the search space keeping track of the decision stack. However, it also prevents the solver from using dynamic decision heuristics, which can lead to worse performance.

All combinations of these options are evaluated, resulting in a total of 8 configurations for CaDiCaL. Furthermore CaDiCaL has the option to only count and not save the found models (`--count` or `-c`). All benchmarks are run with this option enabled, as saving models introduces a significant overhead in used space and IO-operations but does not impact the behavior of the solver itself. (TODO: Add a benchmark showing the impact of saving models? TabularAllSat does save)

6.2 Baseline

As CaDiCaL is a model enumeration implementing all state-of-the-art mechanisms defined and described by Spallitta et al. [3], the natural baseline for comparison is their solver: TabularAllSat. TabularAllSat is a CDCL based solver that uses reason-based propagations to flip variables and shrinks its models when found. Thus, running CaDiCaL with shrinking (`-s`) and reason-based propagation (`-r`) enabled allows a one to one comparison to TabularAllSat.

6.3 Benchmark Sets

Two benchmark sets are used in this evaluation:

- TabularAllSat Benchmarks

A set of benchmarks specifically designed for evaluating All-SAT solvers, introduced by Spallitta et al. [3]. These benchmarks are crafted to have a high number of satisfying assignments and to allow for effective implicant shrinking.

- SATLIB Benchmarks

As implementing the shrinking algorithm through IPASIR-UP is not possible without changing CaDiCaL itself in a non trivial manner, we also evaluate CaDiCaL on a set of benchmarks from SATLIB [?]. These benchmarks are widely used in the SAT community and cover a variety of problem domains. As they are not specifically designed to benefit implicant shrinking, they may provide a different view on CaDiCaLs performance when implicant shrinking is deactivated. The benchmarks used from SATLIB include the planning instances: blocksworld (7 instances) and logistics (3 instances). All interval Series (4 instances) and SAT-encoded Quasigroup (or Latin square) instances (22 instances). This specific set is selected as they are known to have a high number of models and thus, are suitable for evaluating model enumeration but are not especially designed for All-SAT and thus not allow effective shrinking.

6.4 Evaluation Metrics

All benchmarks were run on the Helix cluster of the university of Heidelberg with the following specifications: 20.000 AMD EPYC Milan Processor kernels, around

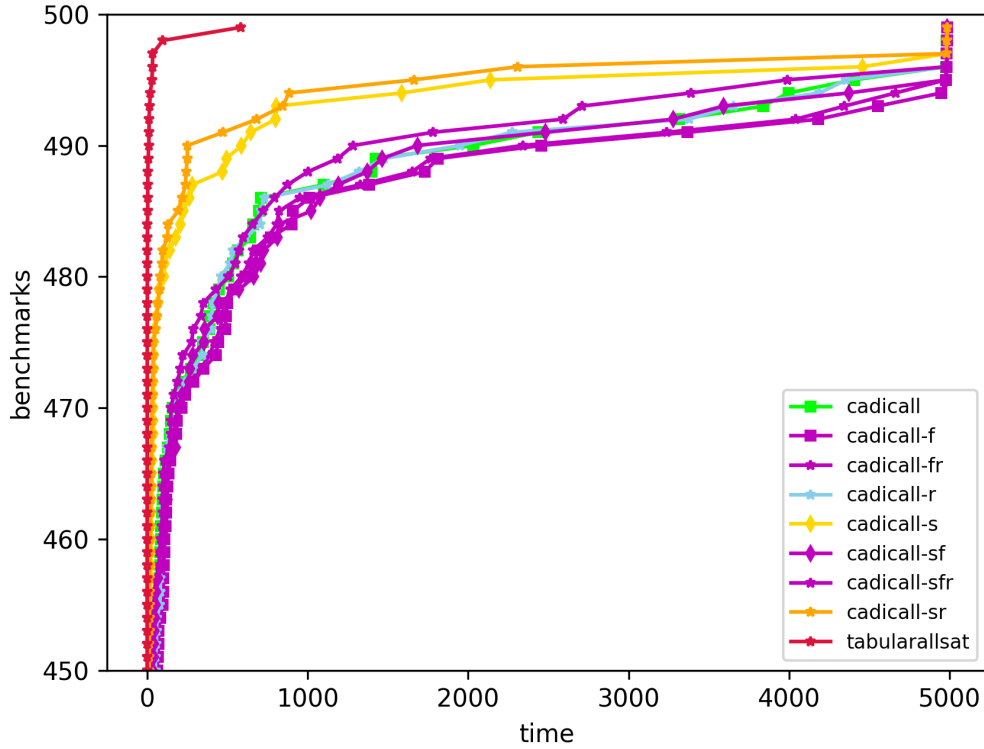


Figure 1: All Benchmarks

100 Terabyte of Space and around 200 NVIDIA Ampere Tensor Core GPUs (A100 und A40). Each benchmarks was run with a time limit of 5000 seconds and a memory limit of 7000 megabyte. The benchmarks compare the time taken to find each model.

6.5 Results

As illustrated in Figure 2, CaDiCall without a fixed variable order exhibits performance comparable to TabularAllSat on the custom benchmark set. Both approaches successfully solve 32 out of 36 instances within nearly identical runtimes. When enabling a fixed variable order, CaDiCall solves 31 instances, i.e., one fewer than in the default configuration. However, for the instances that are solved, the runtime remains largely unchanged and is still comparable to both TabularAllSat and CaDiCall without a fixed variable order. Furthermore, the results indicate that enabling or disabling implicant shrinking and reason-based propagation has no measurable impact on performance for this benchmark set. Specifically, configurations using shrinking (-s) and reason-based propagation (-r) perform identically to those that do not. This behavior is expected, as the custom benchmarks are not designed to benefit from shrinking, and therefore do not allow these techniques to effectively reduce the search space.

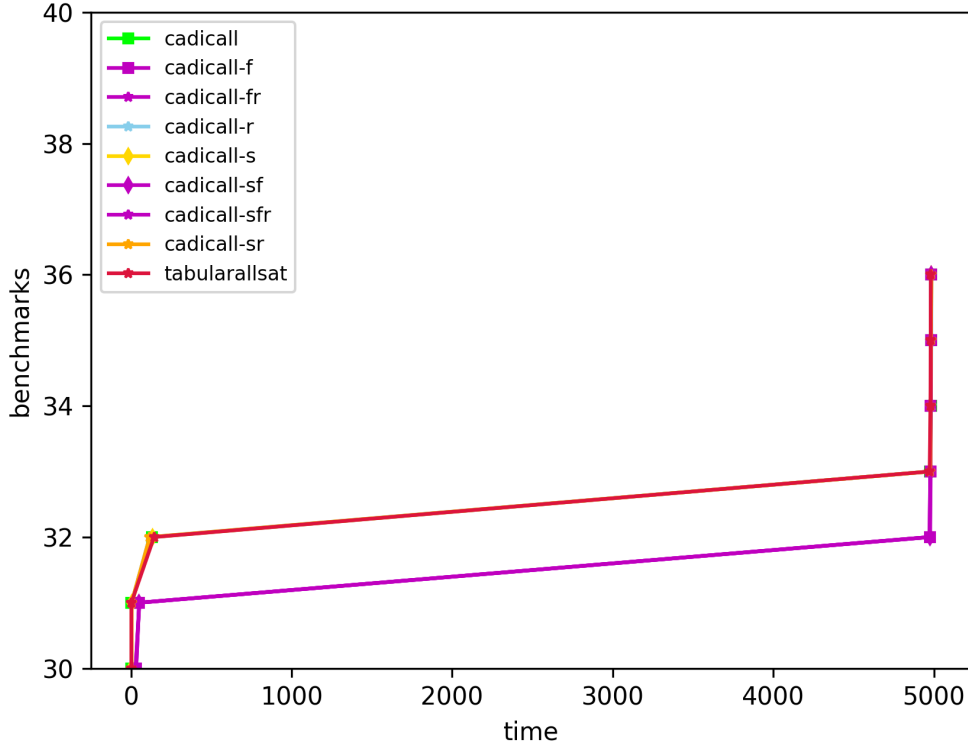


Figure 2: All Custom Benchmarks

An overview of the results on the TabularAllSat benchmark set is shown in Figure 1. The results clearly indicate that TabularAllSat outperforms CaDiCaL on this benchmark suite, both in terms of runtime and the number of solved instances. More specifically, CaDiCaL configured with shrinking (-s), as well as with both shrinking and reason-based propagation (-s -r), exhibits performance comparable to TabularAllSat on the first 480-490 instances. Beyond this range, however, CaDiCaL becomes significantly slower on the subsequent six benchmarks and fails to solve the final three instances. Disabling shrinking leads to a further degradation in performance. This behavior is expected, as the benchmark set is explicitly designed to benefit from effective implicant shrinking, thereby favoring approaches that exploit this technique. Interestingly, when enforcing a fixed variable order (-f), shrinking no longer has a significant impact on the performance of CaDiCaL. A plausible explanation is that, although the benchmarks are constructed to enable effective shrinking, this advantage depends on the solver’s ability to dynamically choose a suitable variable ordering. When the variable order is fixed, the solver may be unable to identify an ordering that facilitates effective shrinking, thereby diminishing its impact on overall performance.

Figure 3 presents a direct comparison between TabularAllSat and the best-performing configuration of CaDiCaL, namely with both shrinking (-s) and reason-

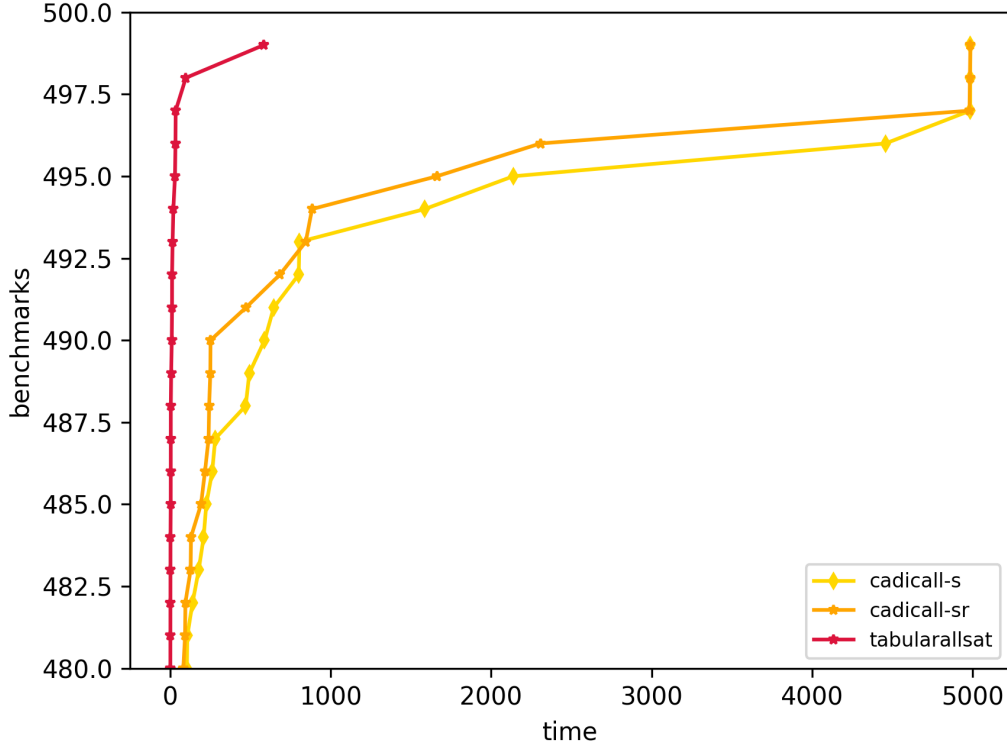


Figure 3: TabularAllSat vs. CaDiCall

based propagation (-r) enabled. This detailed view confirms that TabularAllSat overall outperforms CaDiCall on this benchmark set.

However, CaDiCall is able to match the performance of TabularAllSat on approximately the first 480 benchmarks. Up to instance 487, no significant performance difference can be observed between CaDiCall with both shrinking and reason-based propagation and CaDiCall with only shrinking enabled. Both configurations remain competitive with TabularAllSat in this range.

Beyond this point, CaDiCall without reason-based propagation begins to fall behind. This observation should be interpreted with some caution, as its performance still remains relatively close to the configuration with reason-based propagation. One possible explanation is that the benchmarks are specifically designed to benefit from effective shrinking, and that reason clauses further enhance this effect. Another explanation might be that without reason-based propagation, CaDiCall is more prone to revisiting previously explored regions of the search space again, thereby incurring additional computational overhead.

After approximately 490 benchmarks, both CaDiCall configurations become significantly slower than TabularAllSat and fail to solve the final three instances. These last three benchmarks (BMS_k3_n100_m429_93, BMS_k3_n100_m429_30, and BMS_k3_n100_m429_303) represent the most challenging instances in the benchmark

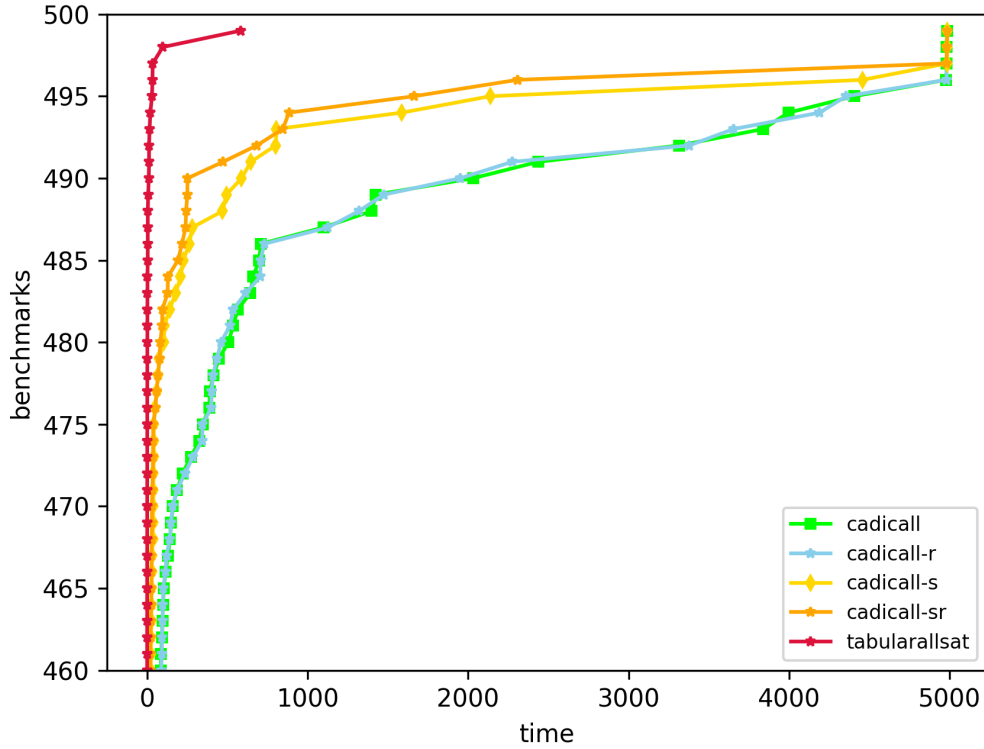


Figure 4: CaDiCAll without fixed variable order

set, with model counts of 19.414.570.229, 151.814.114.272, and 1.802.910.617.456, respectively.

Figure 4 includes the performance of the last CaDiCAll configurations that do not enforce a fixed variable order. Namely, the configurations of CaDiCAll without implicant shrinking but with reason-based propagation (-r) and CaDiCAll without both shrinking and reason-based propagation. These configurations perform significantly worse than the best-performing configuration. Nevertheless, for approximately the first 470 benchmark instances, their performance remains comparable to that of the other CaDiCAll configurations and even to TabularAllSat.

A notable observation is that CaDiCAll without shrinking but with reason-based propagation (-r) performs nearly identically to the configuration without both features. This suggests that reason-based propagation alone does not significantly influence performance when implicant shrinking is disabled. In turn, this supports the hypothesis that the primary benefit of reason-based propagation arises from its interaction with shrinking.

However, this observation should be interpreted with caution. The benchmark set is specifically designed for TabularAllSat, which integrates both implicant shrinking and reason-based propagation. Consequently, the limited effect of reason-based propagation in isolation may be influenced by the structure of the benchmark in-

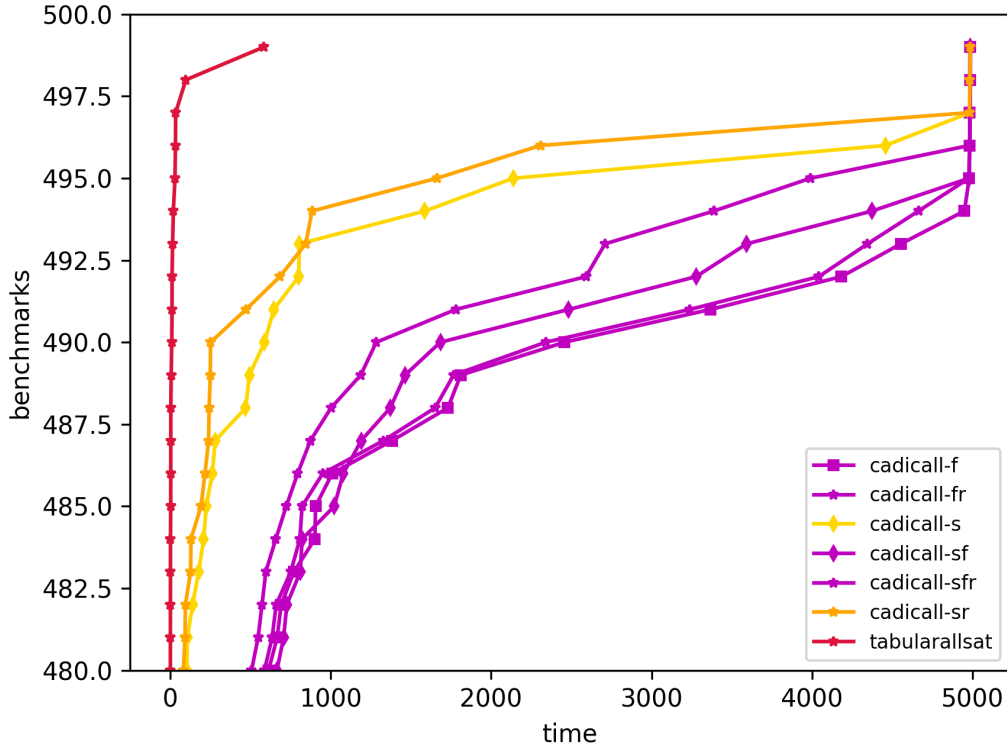


Figure 5: CaDiCAll with fixed variable order

stances. Furthermore, it indicates that omitting reason-based propagation, despite potentially increasing the likelihood of revisiting previously explored regions of the search space and requiring additional backtracking, does not necessarily impose a substantial performance penalty in this setting.

Figure 5 presents the performance of the CaDiCAll configurations that enforce a fixed variable order. Overall, these configurations perform significantly worse than both the best-performing CaDiCAll configuration and TabularAllSat.

Nevertheless, for approximately the first 470 benchmark instances, their performance remains comparable to that of the other CaDiCAll configurations and to TabularAllSat. In addition, they exhibit behavior similar to those CaDiCAll configurations without a fixed variable order in which implicant shrinking is disabled.

While configurations with implicant shrinking still outperform those without, the differences among configurations with a fixed variable order are relatively small. This suggests that the effectiveness of implicant shrinking strongly depends on the solver’s ability to determine a suitable variable ordering dynamically.

This observation is further supported by the results shown in Figure 2, where enforcing a fixed variable order has a less pronounced impact on performance for the custom benchmark set. Taken together, these findings indicate that the TabularAllSat benchmark instances are particularly amenable to shrinking when the solver

is free to choose the variable order, whereas a fixed ordering can substantially limit this advantage.

6.6 Benchmark Conclusion

The experimental results demonstrate that the performance of CaDiCAll is highly dependent on both the chosen configuration and the structural properties of the benchmark instances. On the custom benchmark set, CaDiCAll performs comparably to TabularAllSat, solving nearly the same number of instances within similar runtimes. In this setting, neither implicant shrinking nor reason-based propagation has a measurable impact, which is consistent with the fact that these benchmarks are not designed to benefit from shrinking.

In contrast, the results on the TabularAllSat benchmark set reveal clear performance differences. Here, TabularAllSat consistently outperforms CaDiCAll, particularly on the most challenging instances. While CaDiCAll with implicant shrinking and especially in combination with reason-based propagation remains competitive on a large portion of the benchmarks, its performance degrades significantly on the hardest instances. This highlights the importance of shrinking for efficient model enumeration in instances with large and highly structured solution spaces.

Furthermore, the experiments show that reason-based propagation alone does not significantly improve performance in the absence of shrinking. Its benefits become apparent primarily when combined with shrinking, suggesting a strong interaction between these two techniques. This indicates that reason-based propagation is most effective when it supports the reduction of implicants rather than acting as an independent optimization.

Another key observation concerns the role of variable ordering. Enforcing a fixed variable order consistently degrades performance across the TabularAllSat benchmarks and largely diminishes the benefits of implicant shrinking. This suggests that dynamic variable ordering is crucial for enabling effective shrinking, as it allows the solver to adapt to the structure of the instance. In contrast, the impact of a fixed variable order is less pronounced on the custom benchmarks, further emphasizing that the effectiveness of these techniques is closely tied to the characteristics of the problem instances.

Overall, the results indicate that integrating model enumeration into the solver via IPASIR-UP is a viable approach. However, achieving competitive performance requires careful exploitation of shrinking and solver heuristics. In particular, the combination of implicant shrinking, reason-based propagation, and dynamic variable ordering emerges as the most effective configuration for challenging benchmark instances.

7 Conclusion

...

References

- [1] Stephen A. Cook. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [2] Leonid A. Levin. Universal search problems. *Problems of Information Transmission*, 9(3):265–266, 1973.
- [3] Giuseppe Spallitta, Roberto Sebastiani, and Armin Biere. Disjoint partial enumeration without blocking clauses. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(8):8126–8135, Mar. 2024. doi: 10.1609/aaai.v38i8.28652. URL <https://ojs.aaai.org/index.php/AAAI/article/view/28652>.

Erklärung

Hiermit versichere ich, die vorliegende Arbeit ohne unerlaubte Hilfe und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt zu haben. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen entnommen sind, habe ich als solche kenntlich gemacht. Die eingereichte Masterarbeit wurde weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens. Die elektronische Version der eingereichten Masterarbeit stimmt in Inhalt und Formatierung mit den auf Papier ausgedruckten Exemplaren überein.

Freiburg im Breisgau, den 01.01.2016.